
反思：语言代理与语言强化学习

诺亚·申
东北大学
noahshinn024@gmail.com

费德里科·卡萨诺
东北大学
cassano.f@northeastern.edu

Ashwin Gopinath
麻省理工学院
agopi@mit.edu

Karthik Narasimhan 普林斯顿大学
karthikn@princeton.edu

姚舜禹
普林斯顿大学
shunyuy@princeton.edu

摘要

大型语言模型（LLM）作为目标驱动的代理，越来越多地被用于与外部环境（如游戏、编译器、应用程序接口）进行交互。然而，由于传统的强化学习方法需要大量的训练样本和昂贵的模型微调，因此这些语言代理要想快速、高效地从试错中学习仍然具有挑战性。我们提出的 *Reflexion* 是一个新颖的框架，它不是通过更新权重，而是通过语言反馈来强化语言代理。具体来说，*Reflexion* 代理会对任务反馈信号进行口头反思，然后将自己的反思文本保存在外显记忆缓冲区中，以便在后续试验中做出更好的决策。*Reflexion* 具有足够的灵活性，可纳入各种类型（标量值或自由形式语言）和来源（外部或内部模拟）的反馈信号，并在各种任务（顺序决策、编码、语言推理）中比基线代理获得显著改进。例如，*Reflexion* 在 HumanEval 编码基准测试中达到了 91% 的通过@1 准确率，超过了之前最先进的 GPT-4 的 80%。我们还使用不同的反馈信号、反馈整合方法和代理类型进行了消融和分析研究，并深入探讨了它们对性能的影响。我们在 <https://github.com/noahshinn024/reflexion> 上发布了所有代码、演示和数据集。

1 引言

ReAct [30]、SayCan [1]、Toolformer [22]、HuggingGPT [23]、生成式代理[19]和 WebGPT [17]等最新成果已经证明了建立在大型语言模型（LLM）核心之上的自主决策代理的可行性。这些方法使用 LLM 生成文本和“动作”，这些文本和“动作”可用于 API 调用并在环境中执行。由于这些方法依赖于具有大量参数的大型模型，因此迄今为止，这些方法仅限于使用上下文中的示例来教授代理，因为更传统的优化方案（如梯度下降强化学习）需要大量的计算和时间。

在本文中，我们提出了一种名为“反思”（*Reflexion*）的替代方法，利用口头强化来帮助代理从先前的失败中吸取教训。*Reflexion* 将来自环境的二进制或标量反馈转换为文本摘要形式的口头反馈，然后将其作为额外的上下文添加到下一集的 LLM 代理中。这种自我反思反馈作为一种“语义”梯度信号，为代理提供了一个具体的改进方向，帮助它从之前的错误中吸取教训，从而在任务中表现得更好。这与人类如何反复学习

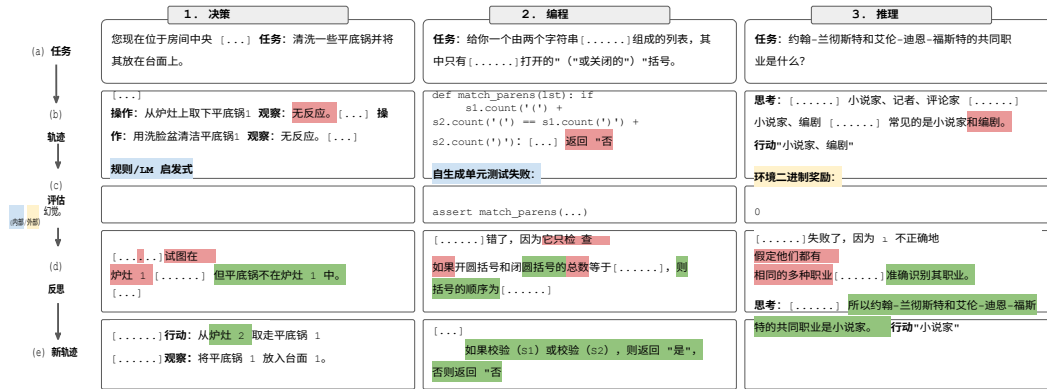


图 1: Reflexion 适用于决策 4.1、编程 4.3 和推理 4.2 任务。

通过反思之前的失败, 为下一次尝试制定改进的攻击计划, 从而以 "一击即中" 的方式完成复杂任务。例如, 在图 1, 中 Reflexion 代理通过尝试、错误和自我反思, 学会优化自己的行为, 以解决决策、编程和推理任务。

生成有用的反思性反馈极具挑战性, 因为这需要很好地理解模型在哪些方面犯了错误 (即学分配问题[25]), 并能够生成包含可操作改进见解的摘要。为此, 我们探索了三种方法--简单的二进制环境反馈、针对常见故障案例的预定义启发式方法, 以及自我评估, 如使用 LLMs 进行二进制分类 (决策) 或自我编写单元测试 (编程)。在所有实施过程中, 评估信号都会被放大为自然语言经验总结, 并存储在长期存储器中。

与更传统的 RL 方法 (如基于策略或价值的学习) 相比, "反思" 有几个优点: 1) 它很轻便, 不需要对 LLM 进行微调; 2) 与标量或矢量奖励相比, "反思" 允许更细致入微的反馈形式 (如有针对性的行动变化), 而标量或矢量奖励则很难进行准确的学分分配; 3) 它允许对先前的经验进行更明确、更可解释的记忆; 4) 它为未来事件中的行动提供了更明确的提示。与此同时, 它也有依赖 LLM 自我评估能力 (或启发式) 的缺点, 而且没有正式的成功保证。不过, 随着 LLM 能力的提高, 我们只希望这种范例能越来越好。

我们进行了以下实验: (1) 决策任务, 测试在长轨迹上的连续行动选择; (2) 推理任务, 测试知识密集型的单步生成改进; (3) 编程任务, 教会代理有效使用外部工具, 如编译器和解释器。在所有三类任务中, 我们发现 Reflexion 代理在决策、推理和编程方面都更胜一筹。更具体地说, 在 12 个迭代学习步骤中, Reflexion 代理在决策 *AtfWorld*[24] 任务上比强大的基线方法提高了 22%, 在 *HotPotQA*[28] 的推理问题上提高了 20%, 在 *HumanEval*[6] 的 Python 编程任务上提高了 11%。

总之, 我们的贡献如下:

- 我们提出了 "反思" (Reflexion) 这一 "口头" 强化的新范式, 它将政策参数化为代理的记忆编码与 LLM 参数的选择。
- 我们探索了 LLMs 中 *自我反思* 的这一新兴特性, 并通过经验表明, 自我反思对于在少量试验中学习复杂任务极为有用。
- 我们介绍 *LeetcodeHardGym*, 这是一个代码生成 RL 健身房环境, 由 19 种编程语言中 40 个具有挑战性的 Leetcode 问题 ("hard-level") 组成。
- 我们的研究表明, Reflexion 在多项任务中的表现都优于强基线, 并在各种代码生成基准测试中取得了最先进的结果。

推理和决策方面的相关工作					
方法	自我提炼	隐藏的制约因素	决策	二进制报酬	内存
自我完善 [15]	✓	✗	✗	✗	✗
光束搜索 [27]	✓	✓	✓	✓	✗
反射 (我们的)	✓	✓	✓	✓	✓

编程方面的工作					
方法	测试执行	测试执行	调试	自制测试	多个语言
阿尔法代码 [14]	✓	✗	✗	✗	✗
CodeT [5]	✓	✗	✗	✗	✗
自我调试 [7]	✓	✓	✗	✗	✗
CodeRL [12]	✓	✓	✗	✗	✗
反射 (我们的)	✓	✓	✓	✓	✓

2 相关工作

推理和决策 自我定义[15]采用了一种自我完善的迭代框架，通过自我评价自主改进生成。这些自我评价和自我完善步骤以给定的任务约束为条件，例如 "如何才能以更积极的方式编写这个生成"。自我定义很有效，但仅限于单代推理任务。Pryzant 等人[21]进行了类似的语义提示书写优化，但也仅限于单代任务。Paul 等人[20]对批评者模型进行了微调，以提供轨迹内的中间反馈，从而改善推理反应。Xie 等人[27]利用对行动的随机波束搜索来执行一种更有效的决策搜索策略，该策略允许代理利用其自我评估组件带来的预见优势。Yoran 等人[31]和 Nair 等人[32]则使用了随机波束搜索。

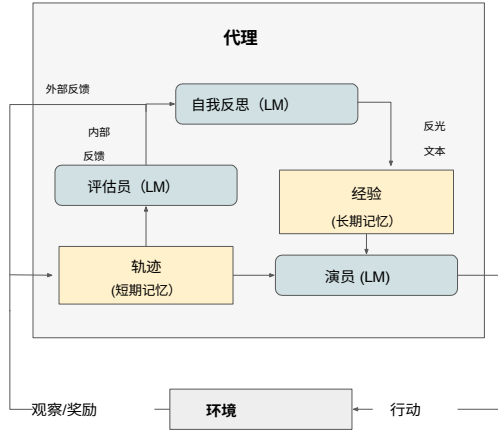
[16]使用判定器模型对多代进行推理。Kim 等人[10]在没有评估步骤的情况下，使用固定步数的重试模式。Goodman [9] 则执行定性评估步骤，对上一代提出优化建议。在本文中，我们展示了这些概念中的几个概念可以通过 *自我反思* 得到增强，从而建立起自我反思经验的持久记忆，使代理能够识别自己的错误，并自我提出经验教训，以便随着时间的推移从错误中吸取教训。

编程 过去和最近的一些工作采用了不同的测试驱动开发或代码调试方法。AlphaCode [14] 在隐藏测试用例上评估了一组生成。CodeT [5] 使用自生成的单元测试对生成的函数实现进行评分。Self-Debugging [7] 采用调试组件，根据代码执行环境的反馈改进现有实现。CodeRL [12]将问题设置在一个 RL 框架中，使用行为批评者设置来调试来自执行环境反馈的程序。AlphaCode、Self-Debugging 和 CodeRL 在修复不太复杂的程序错误方面很有效，但它们都依赖于使 pass@1 资格失效的地面真实测试用例，而没有利用自我反思来弥补错误识别与实现改进之间的差距。CodeT 无法访问隐藏的测试用例，但也没有实施自我学习步骤来改进代码编写。

3 反思：通过口头反思进行强化

我们为 "反思" 开发了一个模块化表述，利用三个不同的模型：一个 *行为者* 模型，用 M_a 表示，它生成文本和动作；一个 *评估者* 模型，用 M_e 表示，它对 M_a 生成的输出进行评分；以及一个 *自我反思* 模型，用 M_s 表示，它生成口头强化提示，以帮助行为者进行自我改进。我们将对这些模型逐一进行详细描述，并随后阐明它们在 "反思" 框架内的协作功能。

Actor Actor 建立在大型语言模型 (LLM) 的基础上，该模型会根据状态观测结果生成必要的文本和动作。与传统的基于策略的 RL 设置类似，我们在 t 时从当前策略 π_θ 中采样一个动作或生成 a_t ，然后接收来自环境 o_t 的观察结果。我们探索了各种行动者模型，包括 Chain of Thought [26] 和 ReAct [30]。通过这些不同的生成模型，我们可以探索以下不同方面



算法 1 通过自我反思强化初始化行为者、评估者、自我反思

```

:
 $M_a, M_e, M_{sr}$ 
初始化策略  $\pi_{\theta}(a_t|s_t), \theta = \{M_a, mem\}$ 
使用  $\pi$  生成初始轨迹  $\theta$ 

用  $M_e$  求出  $\tau_0$ 
使用  $M_{sr}$  生成初始自反射  $sr_0$ 
设置  $mem \leftarrow [sr_0]$ 
设置  $t = 0$ 
while  $M_e$  not pass or  $t < \max \text{ trials}$  do Generate  $\tau_t = [a_0, o_0, \dots, a_i, o_i]$  using  $\pi_{\theta}$  Evaluate  $\tau_t$  using  $M_e$ 
    使用  $M_{sr}$  生成自我反射  $sr_t$ 
    将  $sr_t$  追加到  $mem$ 
    递增  $t$ 
同时结束
返回

```

图 2: (a) Reflexion 的示意图。 (b) Reflexion 强化算法

在 Reflexion 框架内生成文本和动作的过程中，我们对其性能和效果进行了深入分析。此外，我们还添加了一个记忆组件 mem ，为该代理提供额外的上下文。这一调整受到了 Brooks 等人[3]的启发，他们提出了一种使用上下文学习的策略迭代方法。下文将详细介绍这种方法。

评估器 Reflexion 框架的评估器组件在评估行动者生成的输出质量方面起着至关重要的作用。它将生成的轨迹作为输入，并计算出反映其在给定任务环境中表现的奖励分数。定义适用于语义空间的有效价值和奖励函数非常困难，因此我们研究了评价器模型的几种变体。对于推理任务，我们探索了基于精确匹配 (EM) 分级的奖励函数，确保生成的输出与预期的解决方案密切吻合。在决策任务中，我们采用了预先定义的启发式函数，这些函数是根据特定的评估标准量身定制的。此外，我们还尝试使用 LLM 本身的不同实例作为评估器，为决策和编程任务生成奖励。这种多层面的评估器设计方法使我们能够研究不同的策略，为生成的输出结果打分，从而深入了解它们在一系列任务中的有效性和适用性。

自我反思 自我反思模型实例化为一个 LLM，在反思框架中发挥着至关重要的作用，它可以生成口头自我反思，为未来的试验提供有价值的反馈。给定一个稀疏的奖励信号（如二元成功状态（成功/失败））、当前轨迹及其持久记忆 $记忆$ ，自我反思模型就会产生细微而具体的反馈。与标量奖励相比，这种反馈信息量更大，因此会存储在代理的内存 (mem) 中。例如，在一个多步骤决策任务中，当代理收到一个失败信号时，它可以推断出一个特定的行动 a_t 导致了随后的错误行动 $a_{(t+1)}$ 和 $a_{(t+2)}$ 。然后，代理可以口头声明它应该采取不同的行动 a' ，这样就会导致 a'

和 a'_{i+2} ，并将这些经验储存在记忆中。在随后的试验中，代理可以利用过去的 i 通过选择行动 a' ，在 t 时刻调整其决策方法。这种尝试、错误、自我反思和持续记忆的迭代过程，使代理能够利用信息反馈信号，迅速提高其在各种环境中的决策能力。

记忆 "反射"过程的核心要素是短期和长期记忆的概念。在推理过程中，行动者会根据短期和长期记忆做出决定，这与人类在记忆近期细节的同时，还能从长期记忆中回忆起经过提炼的重要经验的方式类似。在 RL 设置中，轨迹历史作为短期记忆，而自我反思模型的输出则存储在长期记忆中。这两种记忆成分共同作用，提供了特定的语境，但也受到经验教训的影响。

这是 Reflexion agents 与其他 LLM 行动选择作品相比的主要优势。

Reflexion 过程 Reflexion 被形式化为 1 中。的迭代优化过程在第一次试验中，行为体通过与环境互动产生轨迹 τ_0 。然后，评估者会产生一个分数 $r_{(0)}$ ，计算公式为 $r_t = M(e)(\tau_0) \circ r_{(t)}$ 只是试验 t 的标量奖励，随着任务特定性能的提高，奖励也会提高。第一次试验后，为了将 r_0 放大为一种反馈形式，供 LLM 用于改进，自我反思模型对 $\{\tau_{(0)}, r_{(0)}\}$ 的集合进行分析，生成一个摘要 $sr_{(0)}$ ，存储在内存 mem 中。如第 3 章所述“反思”的记忆部分对其有效性至关重要。在每次试验 t 之后， sr_t 都会添加 mem 。在实践中，我们用存储经验的最大数量 Ω （通常设置为 1-3）来约束 mem ，以遵守最大上下文 LLM 限制。

4 实验

我们对决策、推理和代码生成任务中的各种自然语言 RL 设置进行了评估。具体来说，我们要求代理在 HotPotQA [28]，中执行基于搜索的问题解答在 AlfWorld [24] 中执行普通家庭环境中的多步骤任务，以及在 HumanEval [6]、MBPP [2] 和 LeetcodeHard（一种新的基准）中执行与解释器和编译器类似的竞争环境下的代码编写任务。最显著的是，Reflexion 在 AlfWorld 中比强基线性能提高了 22%，在 HotPotQA 中提高了 20%，在 HumanEval 中提高了 11%。

4.1 顺序决策ALFWorld

AlfWorld 是一套基于文本的环境，要求代理在各种基于 TextWorld [8] 的交互环境中解决多步骤任务。按照 Yao 等人的研究[30]，我们在 134 个 AlfWorld 环境中运行了代理的六种不同任务，包括寻找隐藏的物体（例如，在抽屉中寻找锅铲）、移动物体（例如，将刀子移到砧板上）以及用其他物体操纵物体（例如，将番茄放入冰箱中冷藏）。我们使用 ReAct[30]作为动作生成器，因为 Yao 等人[30]的研究表明，使用明确的中间思维进行长轨迹决策是成功的。AlfWorld 任务自然需要一个自我评估步骤，因为环境只能发出任务是否完成的信号。为了实现完全自主的行为，我们采用了两种自我评估技术：使用 LLM 的自然语言分类和手写启发式。启发式很简单：如果代理执行相同的操作并得到相同的响应超过 3 个周期，或者在当前环境中执行的操作数量超过 30 个（计划效率低下），我们就会进行自我反思。在基线运行中，如果建议进行自我反思，我们会跳过自我反思过程，重置环境并开始新的试验。在“反思”运行中，代理会通过自我反思来发现错误、更新记忆、重置环境并开始新的试验。为了避免过长的提示窗口超过最大限制，我们将代理的记忆截断为最近的 3 次自我反思（经历）。

为了避免语法错误，我们向代理提供了两个特定领域的几发轨迹。我们在 LLM 中使用了与 Yao 等人[30]使用 GPT-3 时相同的少拍轨迹示例。附录中包括 AlfWorld 任务、ReAct few-shot 提示和 Reflexion 示例。

结果 ReAct + Reflexion 的表现明显优于 ReAct，它利用简单启发式检测幻觉和低效计划，完成了 134 项任务中的 130 项。此外，ReAct+ Reflexion 通过在 12 次连续试验中学习来解决更多任务。在仅使用 ReAct 的方法中，我们发现性能提升在第 6 次和第 7 次试验之间停止。

分析 在基线失败的 AlfWorld 轨迹中，一个常见的错误是，当一个代理认为它拥有了一个物品，但实际上并没有拥有该物品。该代理会在很长的轨迹中执行多个操作，而且无法回溯其操作以发现错误。反思法几乎可以消除所有这些情况，它利用自我反思将漫长的失败轨迹提炼为相关经验，在未来用作“自我提示”。在 AlfWorld 中，长期记忆对代理的帮助主要有两种情况：1) 可以很容易地识别出长轨迹中的早期错误。代理可以提出新的行动选择，甚至是新的长期计划。2) 有太多的

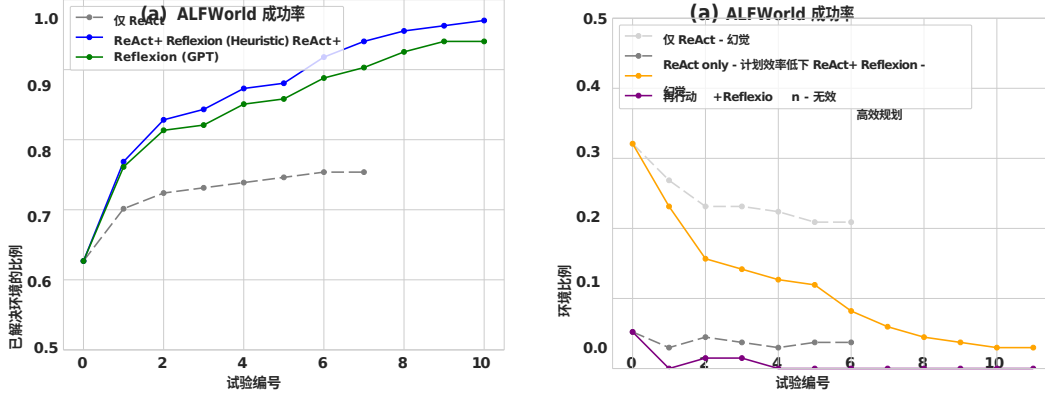


图 3: (a) ALFWorld 在 134 个任务中的表现，显示使用（启发式）和（GPT）自我评估技术进行二元分类时已解决任务的累计比例。(b) 按失败原因对 ALFWorld 轨迹进行分类。

在许多表面/容器中寻找物品。代理可以利用多次试验的经验记忆来彻底搜查一个房间。在 3，中学习曲线表明，学习过程是在多次体验中进行的，这意味着代理成功地平衡了情况 1 和情况 2，表现在前两次试验之间的改进立即激增，然后在接下来的 11 次试验中稳步上升，达到接近完美的性能。另一方面，案例 3 显示，只使用 ReAct 的代理出现了 22% 的幻觉，而且没有长期恢复的迹象。

4.2 推理：火锅QA

HotPotQA[28]是一个基于维基百科的数据集，包含 113k 个问答对，要求代理对内容进行解析，并对多个辅助文档进行推理。为了测试仅推理能力的改进情况，我们实施了反射 + 思维链 (CoT) [26]，分步实现 $Q \rightarrow A$ 和 $Q, C_{gt} \rightarrow A$ ，其中 Q 是问题， C_{gt} 是来自数据集的基本真实上下文， A 是最终答案。由于 CoT 不是一种多步骤决策技术，因此我们将 C_{gt} 交给了代理，这样我们就能在所提供文本的大段内容中分离出推理行为。为了测试需要推理和行动选择的整体问答能力，我们实现了一个 Reflexion + ReAct [30] 代理，它可以使用维基百科应用程序接口检索相关上下文，并使用逐步明确的思维方式推断答案。对于 CoT 的实现，我们使用了 6 次提示；对于 ReAct，我们使用了 2 次提示，而对于自我反思，我们使用了 2 次提示。所有示例均可在附录中找到。

如何稳健地评估自然语言答案是 NLP 领域长期存在的问题。因此，在两次试验之间，我们利用环境对答案进行精确匹配分级，向代理发出二进制成功信号。每次试验后，我们都会利用自我反思循环来放大二进制信号，这与 ALFWorld 中的决策设置 4.1 类似，其内存大小为 3 个经验。

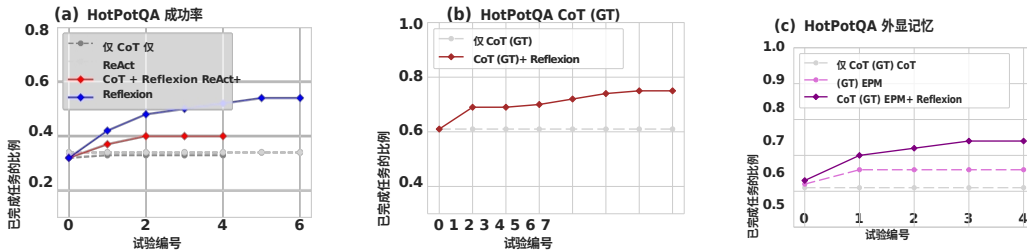


图 4: 思维链 (CoT) 和 ReAct。Reflexion 提高了 100 个 HotPotQA 问题的搜索、信息检索和推理能力。(a) Reflexion ReAct vs Reflexion CoT (b) 仅用于推理的 Reflexion CoT (GT) (c) Reflexion vs episodic memory ablation.

结果 Reflexion 在多个学习步骤中均以显著优势优于所有基准方法。此外，仅 ReAct、仅 CoT 和仅 CoT (GT) 的实现无法在任何任务上实现概率改进，这意味着在温度为 0.7 的情况下，任何基线方法中第一次试验失败的任务都无法在后续试验中得到解决。在 Reflexion 的运行中，我们允许代理积累经验并对失败任务进行重试，直到它在特定任务上连续尝试 3 次失败为止。当然，CoT (GT) 获得了更高的准确率分数，因为它获得了问题的基本真实语境。尽管如此，CoT (GT) 代理仍有 39% 的问题无法正确推断出正确答案，但 Reflexion 可以帮助代理在无法获得基本答案的情况下纠正错误，从而将准确率提高 14%。

分析 我们以 CoT (GT) 为基准方法，进行了一次消融实验，以分离出自我反思步骤在推理中的优势 4。回想一下，CoT (GT) 使用的是 "思维链" (Chain-of-Thought) 推理，并提供了基本真实语境 (ground truth context)，它测试的是长时间语境下的推理能力。接下来，我们通过加入最近的轨迹来添加外显记忆 (EPM) 元素。对于 "反思" 代理，我们实施了标准的自我反思步骤作为最后一关。直观地说，我们通过使用以第一人称书写的语言进行口头解释来测试该代理是否能更有效地进行迭代学习。结果表明，自我反思提高了学习效果，与外显记忆学习优势相比，绝对提升了 8%。这一结果支持了一个论点，即单纯的精炼方法不如自我反思指导下的精炼方法有效。

4.3 编程

我们在 MBPP [2]、HumanEval [6] 和 LeetcodeHardGym (我们的新数据集) 上对 Python 和 Rust 代码编写基准和 Reflexion 方法进行了评估。MBPP 和 HumanEval 衡量的是根据自然语言描述生成函数体的准确性。我们使用基准语言编译器 MultiPL-E [4] 将 HumanEval 和 MBPP 的子集翻译成 Rust 语言。MultiPL-E 是一个小型编译器集合，可用于将 Python 基准问题翻译成其他 18 种语言。我们还包括 Rust 代码生成实验，以证明代码生成的 Reflexion 实现与语言无关，可用于解释型和编译型语言。最后，我们介绍了一个新的基准，即 LeetcodeHardGym，这是一个交互式编程训练场，包含 40 个 Leetcode 高分辨率问题，这些问题都是在 2022 年 10 月 8 日 (即 GPT-4 的预训练截止日期[18])。之后发布的

编程任务为我们提供了一个独特的机会，让我们可以采用更接地气的自我评价方法，如自我生成单元测试套件。因此，我们基于 "反思" 的编程任务实施符合 "通过@1" 准确性报告的要求。为了生成测试套件，我们使用了 "思维链" 提示法

[26]，以生成具有相应自然语言描述的各种广泛测试。然后，我们尝试为每个提议的测试构建一个有效的抽象语法树 (AST)，从而筛选出语法上有效的测试语句。最后，我们从生成的单元测试集合中抽取 n 个测试，生成测试套件 T ，表示为 $\{t_0, t_1, \dots, t_{(n)}\}$ 。我们将 n 设置为最多 6 个单元测试。除了单元测试套件部分，Reflexion 编程代理的学习循环设置与推理和决策代理完全相同，最大内存限制为 1 经验。

基准+ 语言	上一页 SOTA 通行证@1	SOTA Pass@1	反射通道@1
HumanEval (PY)	65.8 (CodeT [5]+ GPT-3.5)	80.1 (GPT-4)	91.0
HumanEval (RS)	-	60.0 (GPT-4)	68.0
甲基溴淘汰计划 (PY)	67.7 (CodeT [5]+ Codex [6])	80.1 (GPT-4)	77.1
甲基溴淘汰计划 (斯普斯卡共和国)	-	70.9 (GPT-4)	75.4
Leetcode Hard (PY)	-	7.5 (GPT-4)	15.0

表 1: 不同模型-策略-语言组合的 Pass@1 准确率。基本策略为单一代码生成样本。所有基于指令的模型均采用零次代码生成。

结果 Reflexion 在 Python 和 Rust 的所有基准测试 (MBPP Python 1。除外) 中的表现均优于所有基准精度，并树立了新的一流标准我们进一步研究了 Reflexion 在 MBPP Python 上的劣势。

分析 我们承认，自我反思的代码生成代理受制于其编写多样化综合测试的能力。因此，在模型生成的测试不完善的情况下

基准+ 语言	基地	反思	TP	FN	FP	TN
HumanEval (PY)	0.80	0.91	0.99	0.40	0.01	0.60
甲基溴淘汰计划 (PY)	0.80	0.77	0.84	0.59	0.16	0.41
HumanEval (RS)	0.60	0.68	0.87	0.37	0.13	0.63
甲基溴淘汰计划 (斯普斯卡共和国)	0.71	0.75	0.84	0.51	0.16	0.49

表 2: HumanEval 和 MBPP 的总体准确率和测试生成性能。对于 Rust, HumanEval 是将 HumanEval Python 中最难的 50 个问题用 MultiPL-E 翻译成 Rust [4]。TP: 单元测试通过, 解决方案通过; FN: 单元测试失败, 解决方案通过; FP: 单元测试通过, 解决方案失败; TN: 单元测试失败, 解决方案失败。

另一方面, 如果模型生成的测试套件不正确, 则有可能所有测试都通过了错误的解决方案, 并导致代码完成时出现假阳性标签[11]。另一方面, 如果模型生成的测试套件编写不正确, 那么部分测试就有可能在正确的解决方案上失败, 从而导致以假阴性代码完成为条件的自我反思生成。考虑到 Reflexion 的实现, 假阴性比假阳性更可取, 因为代理可以使用自我反思来识别错误的测试, 并提示自己保持原始代码完成的完整性。另一方面, 如果无效测试套件返回的是假阳性完成 (所有内部测试用例都通过了, 但实现却不正确), 代理就会过早地报告提交无效。在第 2 章中, 我们对各种条件进行了测量, 以分析超越 pass@1 准确度的性能。此前, 我们曾展示过 Reflexion 在 MBPP Python 上的性能不如基准 GPT-4。在 2, 中我们观察到内部测试执行所产生的假阳性标签 P(未通过@1 生成正确| 测试通过)之间存在明显差异。也就是说, 在通过所有单元测试的情况下, 提交失败的概率。对于 HumanEval 和 MBPP Python, 基线通过@1 准确率比较接近, 分别为 82% 和 80%。不过, MBPP Python 的假阳性测试执行率为 16.3%, 而 HumanEval Python 的假阳性测试执行率仅为 1.4%, 因此总体准确率为 91%。

方法	测试生成	自我反思	Pass@1 (Acc)
基本型号	假的	假的	0.60
测试生成遗漏	假的	正确	0.52
自我反省疏漏	正确	假的	0.60
反思	正确	正确	0.68

表 3: 在 HumanEval Rust 上使用 GPT-4 作为基础模型的 Reflexion 方法中, 各种折衷方法的 Pass@1 准确率 - 50 个最难问题

消融研究 我们在 50 个最难的 HumanEval Rust 问题子集上测试了 Reflexion 测试生成和自我反思合作的复合方法。我们的 Rust 编译器环境提供了详尽的错误日志和有用的调试提示, 因此为折衷方法提供了一个良好的平台。首先, 我们省略了内部测试生成和执行步骤, 这些步骤测试了代理在没有当前实现指导的情况下进行自我反思的能力。结果表明, 第 3 次测试第 3 次测试的准确率为 52%, 而的准确率为 60% (基线), 这表明代理在没有单元测试的情况下无法确定当前实现是否正确。因此, 代理必须参与运行的所有迭代, 而不能选择提前返回, 对实现进行有害的编辑。

接下来, 我们通过省略单元测试套件评估失败后的自然语言解释步骤来测试自我反省的贡献。直观地说, 这要求代理在所有失败的单元测试中将错误识别和实施改进任务结合起来。有趣的是, 与基准运行相比, 改进后的代理并没有提高性能。我们观察到, 测试生成和代码编译步骤能够捕捉语法和逻辑错误, 但实施修复并没有反映出这些迹象。这些实证结果表明, 最近几项提出盲试错调试技术而不进行自我反思的研究, 对于用 Rust 编写复杂程序等难度较大的任务是无效的。

5 局限性

Reflexion 的核心是一种使用自然语言进行策略优化的优化技术。策略优化是一种通过经验改进行动选择的强大方法，但它仍有可能屈服于非最佳局部最小值解决方案。在本研究中，我们将长期内存限制为一个具有最大容量的滑动窗口，但我们鼓励未来的工作利用更先进的结构（如向量嵌入数据库或传统的 SQL 数据库）来扩展 *Reflexion* 的内存组件。具体到代码生成，测试驱动开发在指定精确的输入输出映射方面存在许多实际限制，例如非确定性生成器函数、与 API 交互的不纯函数、根据硬件规格改变输出的函数，或调用并行或并发生态的函数，而这些行为可能难以预测。

6 更广泛的影响

大型语言模型越来越多地被用于与外部环境（如互联网、软件、机器人等）和人类进行交互。我们的工作有可能加强和提高这些代理的自动化程度和工作效率，但同时也扩大了这些代理被滥用的风险。我们认为，这一研究方向需要在安全和伦理方面做出更多努力。

另一方面，强化学习因其黑箱策略和优化设置而受到影响，在这种情况下，可解释性和一致性一直是个难题。我们提出的“言语”强化学习可以解决其中的一些问题，使自主代理变得更加可交互和可诊断。例如，在人类可能难以理解的工具使用情况下，可以对自我反思进行监控，以确保在使用工具之前有正确的意图。

7 结论

在这项工作中，我们提出了 *Reflexion*，这是一种利用语言强化教导代理从过去的错误中学习的方法。我们的经验表明，*Reflexion* 代理通过利用自我反思，明显优于目前广泛使用的决策方法。在未来的工作中，*Reflexion* 可用于采用更先进的技术，这些技术已在传统的 RL 环境中得到深入研究，例如自然语言的价值学习或非策略探索技术。

8 重现性

我们强烈建议其他人在运行自主代码编写实验时使用隔离的执行环境，因为生成的代码在执行前未经验证。

参考资料

- [1] Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., et al. (2022). 做我能做的，而不是我说的: *ArXiv preprint arXiv:2204.01691*.
- [2] Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. (2021). 使用大型语言模型的程序合成. *arXiv preprint arXiv:2108.07732*.
- [3] Brooks, E., Walls, L., Lewis, R. L., and Singh, S. (2022). *arXiv preprint arXiv:2210.03821*.
- [4] Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.-H., Zi, Y., Anderson, C. J., Feldman, M. Q., Guha, A., Greenberg, M., and Jangda, A. (2022). Multipl-e: 可扩展的神经代码生成基准方法.
- [5] Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., and Chen, W. (2022). Codet: *ArXiv preprint arXiv:2207.10397*.

- [6] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021).评估基于代码训练的大型语言模型。 *arXiv preprint arXiv:2107.03374*.
- [7] Chen, X., Lin, M., Schärli, N., and Zhou, D. (2023). 教大型语言模型自调试。 *arXiv preprint arXiv:2304.05128*.
- [8] Côté, M.-A., Kádár, A., Yuan, X., Kybartas, B., Barnes, T., Fine, E., Moore, J., Hausknecht, M., El Asri, L., Adada, M., et al. (2019).文本世界：基于文本的游戏学习环境。 In *Computer Games: 7th Workshop, CGW 2018, Held in Conjunction with the 27th International Conference on Artificial Intelligence, IJCAI 2018, Stockholm, Sweden, July 13, 2018, Revised Selected Papers 7*, pages 41-75.Springer.
- [9] Goodman, N. (2023).元提示： *Noahgood- man.substack.com*.
- [10] Kim, G., Baldi, P., and McAleer, S. (2023). 语言模型可解决计算机任务。 *arXiv 预印本 arXiv:2303.17491*.
- [11] Lam, W., Winter, S., Wei, A., Xie, T., Marinov, D., and Bell, J. (2020). 片状测试的大规模纵向研究。 *Proc.ACM Program.4(OOPSLA)*.
- [12] Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. H. (2022).Coder1：通过预训练模型和深度强化学习掌握代码生成。 *神经信息处理系统进展*，35:21314-21328。
- [13] Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. (2023).StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.
- [14] Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. (2022).用alphacode生成竞赛级代码。 *科学*，378 (6624)：1092-1097。
- [15] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y., et al. (2023).Self-refine： *arXiv preprint arXiv:2303.17651*.
- [16] Nair, V., Schumacher, E., Tso, G., and Kannan, A. (2023)。Dera：Dera: *Enhancing large language model completions with dialog-enabled resolving agents. ArXiv preprint arXiv:2303.17071*.
- [17] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. (2021).Webgpt： *ArXiv preprint arXiv:2112.09332*.
- [18] OpenAI (2023).Gpt-4技术报告。 *ArXiv*.
- [19] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023).Generative agents： *arXiv preprint arXiv:2304.03442*.
- [20] Paul, D., Ismayilzada, M., Peyrard, M., Borges, B., Bosselut, A., West, R., and Faltings, B.(2023).Refiner： *ArXiv preprint arXiv:2304.01904*.
- [21] Pryzant, R., Iter, D., Li, J., Lee, Y. T., Zhu, C., and Zeng, M. (2023)。使用 "梯度下降 "和波束搜索的自动及时优化。 *arXiv 预印本 arXiv:2305.03495*.
- [22] Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023).Toolformer： *ArXiv preprint arXiv:2302.04761*.
- [23] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. (2023).Hugginggpt： *ArXiv preprint arXiv:2303.17580*.

- [24] Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., and Hausknecht, M. (2021). ALFWorld: 为交互式学习调整文本和嵌入式环境。 *国际学习表征会议 (ICLR) 论文集*。
- [25] Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: 导论*。麻省理工学院出版社, 第二版。
- [26] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., and Zhou, D. (2022)。思维链提示激发大型语言模型中的推理。 *arXiv preprint arXiv:2201.11903*。
- [27] Xie, Y., Kawaguchi, K., Zhao, Y., Zhao, X., Kan, M.-Y., He, J., and Xie, Q. (2023). *ArXiv preprint arXiv:2305.00633*。
- [28] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. (2018)。HotpotQA: 用于多样化、可解释多跳问题解答的数据集。 *自然语言处理实证方法会议 (EMNLP)*。
- [29] Yao, S., Chen, H., Yang, J., and Narasimhan, K. (preprint). 网店: 利用基础语言代理实现可扩展的真实世界网络交互。见 *ArXiv*。
- [30] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: 语言模型中推理与行动的协同。 *国际学习表征会议 (ICLR)*。
- [31] Yoran, O., Wolfson, T., Bogin, B., Katz, U., Deutch, D., and Berant, J. (2023)。通过多思维链的元推理回答问题。 *arXiv preprint arXiv:2304.13007*。

A 用其他模型进行评估

我们进一步研究了试错法解决问题在不同强度模型中的适用性。我们发现，指定自我修正的能力是更强、更大的模型所具有的新兴品质。

方法	通过率@1 准确率（8 次试验的平均值）	1 次通过精度（标准）
基线	0.26	0.00481
反思	0.26	0.00305

表 4: 使用 starchat-beta [13], 。HumanEval Python 的 Pass@1 准确率

模型	基准精度	反射精度
CoT (GT)+ text-davinci-003	0.60	0.77
CoT (GT)+ gpt-3.5-turbo	0.57	0.71
CoT (GT)+ gpt-4	0.68	0.80
ReAct+ text-davinci-003	0.30	0.55
ReAct+ gpt-3.5-turbo	0.26	0.38
ReAct+ gpt-4	0.39	0.51

表 5: 在 100 个 HotPotQA 中使用各种模型的 Pass@1 准确率。

B 决策

环境你正身处房间中央。快速环顾四周，你看到一张床 1、一张书桌 2、一张书桌 1、一个抽屉 6、一个抽屉 5、一个抽屉 4、一个抽屉 3、一个抽屉 2、一个抽屉 1、一个垃圾桶 1、一个

洗衣房 1、保险箱 1、架子 6、架子 5、架子 4、架子 3、架子 2 和架子 1。

任务：用台灯检查杯子。

1号试验

> 想：为了完成任务，我需要找到并拿走一个杯子，然后找到并使用一盏台灯。

> 想一想首先我要找到一个杯子。马克杯更有可能出现在抽屉（1-6）、书桌（1-2）、书架（1-6）、垃圾桶（1）、洗衣篮（1）中。我可以从抽屉 1 开始逐个检查。

> 前往第 1 抽屉

抽屉 1 关闭。

...

> 打开抽屉 6

打开抽屉 6。抽屉 6 打开了。您在其中看到了一个钥匙扣 2。

> 转到 1 号桌

在桌子 1 上，你可以看到一张信用卡 3、一盏台灯 1、一台笔记本电脑 2、一个杯子 1、一支钢笔 1 和一支铅笔 1。

> 想：现在我找到了一个杯子 (1)。接下来，我要拿走它。

> 从 1 号桌拿杯 1

你从桌子 1 上拿起杯子 1。

> 想：现在我拿一个杯子 (1)。接下来，我要找一盏台灯。台灯更有可能出现在书桌 (1-2)、餐具柜 (1-2)、书架 (1-6)、床 (1)、抽屉 (1-6)。我可以从书桌 1 开始逐个检查。

> 转到 1 号桌 什么

都没发生。

> 转到 2 号桌

在桌子 2 上，你可以看到闹钟 1、碗 1、杯子 3、铅笔 3 和铅笔 2。

> 想：现在我找到了一盏台灯 (1)。接下来，我需要使用它。

> 使用台灯 1

> 使用台灯 1 什么

都不会发生。状态

：失败

反思：在这个环境中，我的计划是找到一个杯子，然后找到并使用台灯。但是，任务要求用台灯检查杯子。我应该先找台灯，然后再找杯子。我注意到台灯是在 1 号桌子上找到的。在下一试验中，我将去 1 号桌找到台灯，然后寻找马克杯并用台灯检查。

2号试验

> 转到 1 号桌

在桌子 1 上，你可以看到一张信用卡 3、一盏台灯 1、一台笔记本电脑 2、一个杯子 1、一支钢笔 1 和一支铅笔 1。

> 想：为了完成任务，我需要找到并拿走一个杯子，然后找到并使用一盏台灯。

> 从 1 号桌拿杯 1

你从桌子 1 上拿起杯子 1。

> 想：为了完成任务，我需要找到并拿走一个杯子，然后找到并使用一盏台灯。

> 使用台灯 1

您打开台灯 1。状态：成功

图 5：[上图] 一个 AlfWorld 轨迹，其中代理因规划效率低下而失败。在反思中，代理认识到它应该先找台灯再找

杯子，而不是先找杯子再找台灯。[下图] 代理能够纠正其推理轨迹，并以简洁的方式执行一系列行动。

B.1 网上商店限制

在第 5 节中，我们简要说明了 Reflexion 在克服局部极小值选择方面的困难，这些选择需要极富创造性的行为才能摆脱。我们在 WebShop [29] 的实验中观察到了这一缺陷。WebShop 是一个基于网络的问题解决基准测试，测试代理如何根据客户的要求浏览电子商务网站，查找并购买产品。我们在 100 个环境中测试了一个双镜头 ReAct + Reflexion 代理。然而，仅进行了四次试验后，我们就终止了运行，因为代理没有显示出改进的迹象。此外，该代理在尝试失败后也没有产生有益的、直观的自我反思。我们的结论是，Reflexion 无法解决需要大量多样性和探索性的任务。在 AlfWorld 中，代理能够充分探索新环境，因为从观察结果中可以看到允许采取的行动。在 HotPotQA 中，代理面对的是类似的 WebShop 搜索查询任务，但由于维基百科文章的搜索空间更加多样化，对搜索查询的精确度要求较低，因此代理的任务更加成功。电子商务搜索引擎的一个常见问题是正确处理自然语言搜索解释中的歧义。因此，WebShop 提出的任务需要 Reflexion 代理做出非常多样和独特的行为。

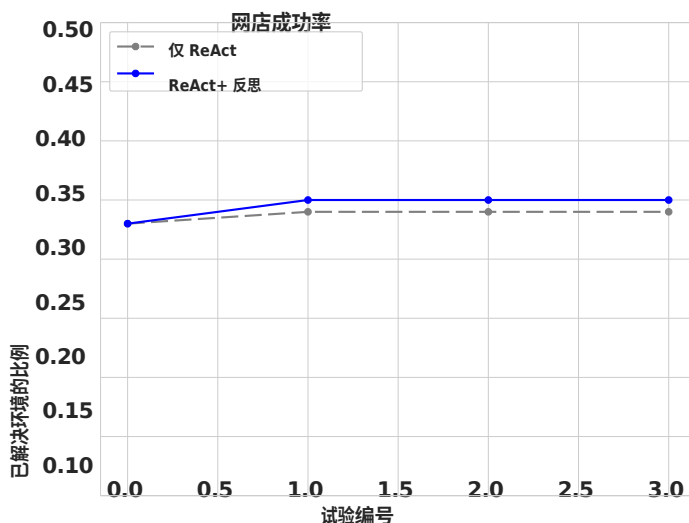


图 6: Reflexion 与 React 在 WebShop 上处理 100 个客户购物请求的性能对比。ReAct + Reflexion 的性能未能明显优于 ReAct。

C 编程

由于 LLM 接受过大量的对话培训，因此在对 LLM 调用进行编程时需要严格遵守只能生成函数体的指令。下面报告了几个编程示例，其中的指令用蓝色和模板标出。请访问 <https://github.com/noahshinn024/reflexion> 查看完整的实现过程。

C.1 编程函数实现示例 (HumanEval Python)

函数签名示例：

```
1 def min Sub Array Sum ( nums ) :  
2     """  
3     给定一个整数数组 nums ， 求其中任何一个数组的最小和。  
4     非- 空子- 数组。  
5     示例  
6     min Sub ArraySum ([2 , 3 , 4 , 1 , 2 , 4]) == 1
```

```

7 min Sub ArraySum ([-1, -2, -3]) == -6
8 """

```

函数体实现示例：

```

1 min_sum= float (' inf ')
2 for i in range ( len ( nums )): current_sum = 0
3     for j in range ( i, len ( nums )):
4         current_sum+= nums [ j]
5         if current_sum< min_sum : min_sum=
6             current_sum
7 返回 min_sum
8

```

C.2 反思 演员指导

你是一名 Python 写作助手。我们会给你一个函数的先前实现、一系列单元测试结果以及你对先前实现的自我反思。请在回复中仅使用改进后的函数体，对下面的内容进行必要的修改。请勿在回复中包含签名。回复的第一行应缩进 4 个空格，以便在语法上与用户提供的签名一致。用户会给您提供一些示例。

反思演员世代相传的形式：

（教学）

（功能实现）（单元测试反馈）

（自我反省）

（下一次函数植入的指令）

完整示例见 <https://github.com/noahshinn024/reflexion>

C.3 反思 自我反思的指导和范例

你是一名 Python 写作助手。我们会给你一个函数的先前实现、一系列单元测试结果以及你对先前实现的自我反思。请在回复中仅使用改进后的函数体，对下面的内容进行必要的修改。请勿在回复中包含签名。回复的第一行应缩进 4 个空格，以便在语法上与用户提供的签名一致。用户会给您提供一些示例。

自我反思 按照表格进行自我

反思：

（教学）

（功能实现）（单元测试反馈）

C.4 反射编程 无自反射消融示例

反射 无自我反射消融演员世代遵循的形式：（说明）

（功能实现）（单元测试反馈）

（自我反省）—

（下一次函数植入的指令）

C.5 反射编程 无测试生成烧蚀示例

反射 无测试代烧蚀演员代遵循的形式：（说明）

(功能实现) —(单元测试反馈)—
(自我反省)
(下一次函数植入的指令)

D 推理

D.1 完整示例

试验 #1: 问题: Grown-Ups 的主演最著名的角色是 "Allo 'Allo!" 中的哪个角色?

思考 1: 我需要搜索 "成年人" 和 "儿童"。

"Allo'Allo!" 中主演的演员, 然后查找他在 "Allo'Allo!" 中最著名的角色。

行动 1: 搜索[成年人]

观察 1: 《长大成人》是英国 BBC 于 1980 年拍摄的一部电视电影, 由迈克-利 (Mike Leigh) 设计并执导。影片由莱斯莉-曼维尔、菲利普-戴维斯、布伦达-布莱辛、珍妮-杜维茨基、林赛-邓肯和山姆-凯利主演。该片由 Robin Sales 剪辑, Louis Marks 为 BBC 制作, 最初于 1980 年 11 月 28 日在 BBC 2 频道播出。

思考 2: 《长大成人》由莱斯莉-曼维尔、菲利普-戴维斯、布伦达-布莱辛、珍妮-杜维茨基、林赛-邓肯和山姆-凯利主演。我需要搜索 "Allo 'Allo!", 找出哪位演员因哪个角色而最为人熟知。

操作 2: 搜索["Allo 'Allo! "]

意见 2: 找不到 ["Allo 'Allo! "]。相似: ["Allo 'Allo! ", "Allo 'Allo! ' 人物列表", 'Gorden Kaye', "Allo 'Allo! ' (系列 2) ", 'Allô, allô', 'Francesca Gonshaw', "Allo 'Allo! ' (系列 4) ", 'Kim Hartman', "Allo 'Allo! "

思考 3: 要查找因在 "Allo 'Allo!" 中扮演哪个角色而最为人熟知的演员, 我可以搜索 Gordon Kaye。

行动 3: 搜索[戈登-凯]

观点 3: 戈登-欧文-凯 (Gordon Irving Kaye, 1941 年 4 月 7 日-2017 年 1 月 23 日), 职业名戈登-凯, 英国演员, 因在电视喜剧系列剧《Allo 'Allo!

思考 4: 戈登-凯因在电视喜剧系列剧 "Allo 'Allo!" 中扮演好色的咖啡馆老板 René Artois 而闻名。所以答案是

Rene Artois。

行动 4: 完成[Rene Artois] 观察 4: 答案不正

确

反思: 我搜索了错误的节目名称 "Allo 'Allo!", 结果没有任何结果。我应该搜索该剧的主角 Gordon Kaye, 找到他在剧中最著名的角色。

试验 #2: 问题: Grown-Ups 的主演最著名的角色是 "Allo 'Allo!" 中的哪个角色?

思考 1: 我需要搜索曾担任过.....的演员。

'Allo'Allo!' 中最著名的角色, 并找出他们最著名的角色。

行动 1: 搜索[成年人]

观察 1: 《长大成人》是英国 BBC 于 1980 年拍摄的一部电视电影, 由迈克-利 (Mike Leigh) 设计并执导。影片由莱斯莉-曼维尔、菲利普-戴维斯、布伦达-布莱辛、珍妮-杜维茨基、林赛-邓肯和山姆-凯利主演。该片由 Robin Sales 剪辑, Louis Marks 为 BBC 制作, 最初于 1980 年 11 月 28 日在 BBC 2 频道播出。

思考 2: 该段没有提到因在 "Allo 'Allo!" 中扮演角色而闻名的演员。我需要搜索该演员的名字。操作 2: 搜索[萨姆-凯利]

观察2: 罗杰-迈克尔-凯利 (1943年12月19日-2014年6月14日), 艺名山姆-凯利, 英国演员, 曾出演过电影、电视、广播和戏剧。他最著名的角色是《Allo 'Allo!

思考 3: 萨姆-凯利因在《'Allo 'Allo!》中扮演汉斯-盖林船长而闻名, 所以答案是汉斯-盖林船长。

行动 3: 完成[汉斯-盖林船长] 观察 3: 答案正

确

图 7: 在同一环境和任务中进行的两次 HotPotQA 试验。Reflexion+ ReAct 代理利用自我反思为下一次试验确定更好的搜索方法。

D.2 思维链+ 反思

试验 #1: 问题: 约翰-兰彻斯特和艾伦-迪恩-福斯特从事什么职业?

思考 1: 让我们逐步思考。约翰-兰奇埃斯特是小说家、记者和评论家。艾伦-迪恩-福斯特是小说家和编剧。因此, 约翰-兰彻斯特和艾伦-迪恩-福斯特的共同职业是小说家和编剧。

行动 1: 完成[小说家和编剧] 观察 1: 答案不正确

反思: 我对约翰-兰彻斯特和艾伦-迪恩-福斯特的共同职业的推理失败了, 因为我错误地认为他们都有相同的职业。今后, 在做这道题时, 我应该集中精力更好地研究两位作者的个人背景, 以确保我能准确地确定他们的职业。此外, 我还应该考虑他们可能有多种共同职业的可能性。

试验 #2: 问题: 约翰-兰彻斯特和艾伦-迪恩-福斯特从事什么职业?

思考 1: 让我们逐步思考。约翰-兰奇埃斯特是小说家、记者和财经记者。艾伦-迪恩-福斯特是小说家和编剧。因此, 约翰-兰彻斯特和艾伦-迪恩-福斯特的共同职业是小说家。

行动 1: 完成[小说家]

观察结果 1: 答案正确

D.3 HotPotQA 思维链 (GT)+ 反思

背景: 怀特普莱恩斯战役是美国革命战争纽约和新泽西战役中的一场战役, 于 1776 年 10 月 28 日在纽约怀特普莱恩斯附近打响。在撤退之后...

试验 #1: 问题什么是一系列战斗

在革命战争期间, 为了争夺新泽西州的控制权

纽约市和新泽西州在以下方面展开了斗争

1776 年 10 月 28 日, 纽约州怀特普莱恩斯附近?

思考 1: 问题问的是 1776 年 10 月 28 日在纽约州怀特普莱恩斯 (White Plains) 附近为争夺纽约市和新泽西州的控制权而发生的一场革命战争。根据上下文, 我知道这是白原战役。

行动 1: 完成[白原战役] 观察 1: 答案不正确

反思: 在反思我提供的错误答案后, 我意识到我可能没有提供足够的背景信息来准确回答问题。问题要求回答一系列战役, 但我只提供了一场战役的名称。为了今后更好地回答这个问题, 我将确保提供更多的背景信息, 例如战役的名称, 即纽约和新泽西战役, 以及这是一系列战役的事实。此外, 我还将确保提供战役的日期和地点, 以确保我的答案全面准确。

试验 #2: 问题: 1776 年 10 月 28 日, 为了争夺纽约市和新泽西州的控制权, 革命战争期间在纽约州怀特普莱恩斯附近发生了一系列战斗, 这些战斗是什么?

思考 1: 问题问的是在革命战争期间, 1776 年 10 月 28 日在纽约州怀特普莱恩斯附近为争夺纽约市和新泽西州的控制权而发生的一系列战役。根据上下文, 我知道这一系列战役就是纽约和新泽西战役。

行动 1: 完成[纽约和新泽西战役]

观察结果 1: 答案正确

D.4 HotPotQA 外显记忆 (EPM) 消融提示

D.4.1 (EPM) 思维链+ 反思

试验 #1: 问题: 乔尼-克雷格 (Jonny Craig) 和皮特-多尔蒂 (Pete Doherty) 中哪一个曾是更多乐队的成员

?

思考 1: 让我们逐步思考。乔尼-克雷格曾是六支乐队的成员, 包括 Emarosa、Dance Gavin Dance 和 Slaves。皮特-多尔蒂 (Pete Doherty) 曾是七支乐队的成员, 其中包括 The Libertines、Babyshambles 和 The Puta Madres。因此, 皮特-多尔蒂曾是更多乐队的成员。

行动 1: 完成[Pete Doherty] 观察 1: 答案不正

确

试验 #2: 问题: 乔尼-克雷格 (Jonny Craig) 和皮特-多尔蒂 (Pete Doherty) 中, 谁参加过的乐队更多

?

思考 1: 让我们逐步思考。我将重新搜索两位音乐人过去和现在的乐队, 以确保准确比较他们的乐队成员身份。经过研究, 我发现乔尼-克雷格曾是七支乐队的成员, 包括 Emarosa、Dance Gavin Dance 和 Slaves。皮特-多尔蒂曾是七支乐队的成员, 包括 The Libertines、Babyshambles 和 The Puta Madres。乔尼-克雷格 (Jonny Craig) 曾是更多乐队的成员。

行动 2: 完成[Jonny Craig] 观察 2: 答案正

确

反思: 我对哪位音乐家曾加入过更多乐队的推理失败了, 因为我没有考虑到乔尼-克雷格 (Jonny Craig) 过去曾加入过更多的乐队, 尽管皮特-达赫迪 (Pete Doherty) 目前加入的乐队更多。今后, 当我尝试回答这个问题时, 我应该重点研究这两位音乐家过去和现在的乐队, 以确保我能够准确地比较他们的乐队成员身份。

D.4.2 (EPM) 思维链 (GT)+ 反思

背景 哈里-巴哈杜尔-巴斯内特是尼泊尔政治家。他是 Rastriya Janashakti 党对外关系部负责人。巴斯内特拥有工程学硕士学位。

理学硕士 (拉丁语: "Magister Scientiae"; 缩写 MS、M.S.、MSc、M.Sc.、MSci、M.Sci.、ScM、Sc.M.、SciM 或 Sci.M.) 是科学领域的硕士学位。

试验 #1: 问题: Rastriya Janashakti 党对外关系部负责人拥有哪个领域的学位, 可缩写为 MS、M.S. 或 ScM?

?

(上一个轨迹)

思考 1: 问题问的是 Rastriya Janashakti 党对外关系部负责人拥有哪个专业的学位。根据上下文, 我知道该学位是理学硕士学位, 通常授予科学、工程和医学方面的研究。

行动 1: 完成[科学、工程和医学]项目

意见 1: 答案不正确

反思: 第一次试验, 我误解了问题。我以为问题要求而不是一个具体的学位。我从自己的错误中吸取了教训, 在第二次试验中, 我更仔细地研究问题, 才明白它是在询问一个具体的研究领域。

试验 #2: 问题: Rastriya Janashakti 党外交关系部部长拥有哪个领域的学位, 可缩写为 MS、M.S. 或 ScM?

(上一个轨迹)

思考 1: 问题问的是 Rastriya Janashakti 党对外关系部负责人拥有哪个专业的学位。根据上下文, 我知道该学位是理学硕士学位, 通常授予科学、工程和医学方面的研究。然而, 问题问的是研究领域, 而不是学位。根据上下文, 我知道 Hari Bahadur Basnet 拥有工程学硕士学位。

行动 1: 完成[工程] 观察 1: 答案正确